

OBJECT ORIENTED PROGRAMMING USING C++

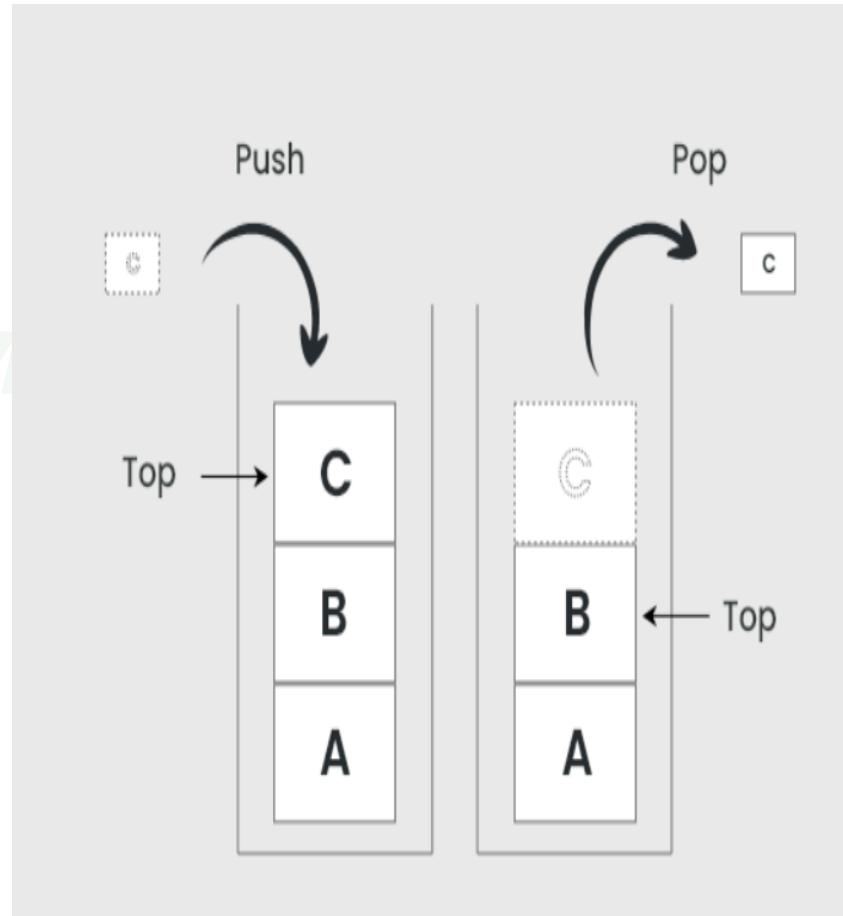


Study Materials

Stack:

- A stack is a linear data structure that follows the Last In First Out (LIFO) principle.
- **Basic Operations:**
- **push()** to insert an element into the stack
- **pop()** to remove an element from the stack
- **top()** Returns the top element of the stack.
- **isEmpty()** returns true if the stack is empty else false.
- **size()** returns the size of the stack.

Stack:



Expressions:

- Infix Expression:
 - Operators are placed between operands (e.g., $A + B$).
- Prefix Expression:
 - Operators precede their operands (e.g., $+ A B$).
- Postfix Expression:
 - Operators follow their operands (e.g., $A B +$).

Infix Expression:

- Definition: An expression where operators are placed between operands (e.g., $A + B$).
- **Characteristics:**
 - Requires parentheses to dictate precedence.
 - Evaluation order is determined by operator precedence and associativity.
 - Example:
 - Infix: $A + B * C$

Prefix Expression:

- Definition: An expression where operators precede their operands (e.g., + A B).
- **Characteristics:**
 - No need for parentheses to dictate precedence.
 - Evaluated from right to left.
 - Conversion: $A + B * C \rightarrow + A * B C$

Postfix Expression:

- Definition: An expression where operators follow their operands (e.g., A B +).
- **Characteristics:**
 - Also does not require parentheses.
 - Evaluated from left to right.
 - Conversion: $A + B * C \rightarrow A B C * +$

Converting Infix to Prefix and Postfix:

- Conversion Techniques:
- **Infix to Postfix:**
 - Use a stack to hold operators.
 - Follow the Shunting Yard algorithm.
- **Infix to Prefix:**
 - Reverse the infix expression.
 - Convert to postfix and then reverse the result.
- **Example:**
 - Infix: $A + B * C$
 - Postfix: $A B C * +$
 - Prefix: $+ A * B C$

Evaluating Postfix Expressions:

- **Algorithm:**
- Read the expression from left to right.
- If an operand is found, push it onto the stack.
- If an operator is found, pop the required number of operands from the stack, perform the operation, and push the result back onto the stack.

Example:

- **Example:**
- Postfix: $3\ 4\ +\ 2\ *\ 7\ /\$
- **Steps:**
 - Push 3 and 4
 - Pop 3 and 4, compute $3 + 4 = 7$, push 7
 - Push 2, compute $7 * 2 = 14$, push 14
 - Push 7, compute $14 / 7 = 2$

Applications of Stacks:

- Function calls: Stacks are used to keep track of the return addresses of function calls, allowing the program to return to the correct location after a function has finished executing.
- Recursion: Stacks are used to store the local variables and return addresses of recursive function calls, allowing the program to keep track of the current state of the recursion.
- Expression evaluation: Stacks are used to evaluate expressions in postfix notation (Reverse Polish Notation).
- Syntax parsing: Stacks are used to check the validity of syntax in programming languages and other formal languages.
- Memory management: Stacks are used to allocate and manage memory in some operating systems and programming languages.

Advantages of Using Stacks:

- **Simplicity:** Stacks are a simple and easy-to-understand data structure, making them suitable for a wide range of applications.
- **Efficiency:** Push and pop operations on a stack can be performed in constant time ($O(1)$), providing efficient access to data.
- **Last-in, First-out (LIFO):** Stacks follow the LIFO principle, ensuring that the last element added to the stack is the first one removed.
- **This behavior is useful in many scenarios,** such as function calls and expression evaluation.
- **Limited memory usage:** Stacks only need to store the elements that have been pushed onto them, making them memory-efficient compared to other data structures.

Disadvantages of Stacks:

- **Limited access:** Elements in a stack can only be accessed from the top, making it difficult to retrieve or modify elements in the middle of the stack.
- **Potential for overflow:** If more elements are pushed onto a stack than it can hold, an overflow error will occur, resulting in a loss of data.
- **Not suitable for random access:** Stacks do not allow for random access to elements, making them unsuitable for applications where elements need to be accessed in a specific order.
- **Limited capacity:** Stacks have a fixed capacity, which can be a limitation if the number of elements that need to be stored is unknown or highly variable.